

A High Throughput and Low Power Radix-4 FFT Architecture

Soumak Mookherjee, Linda DeBrunner, Victor DeBrunner

Electrical & Computer Engineering, Florida State University

Abstract—In this paper, a high throughput and low power architecture for 256-point FFT processor is proposed which is suitable for both high performance and low power applications. The proposed architecture is based on Radix-4 algorithm. We choose pipelined Multi-path Delay Commutators (MDC) for our design. Two separate datapaths are used in this architecture so that it can process eight inputs in parallel. Thus, the throughput is increased by eight times while achieving 100% hardware utilization. Power consumption of this architecture is shown to be about 50% less than a regular Radix-4 MDC structure for a same throughput. We implement our design in Xilinx FPGA Virtex 5 and compare with regular R4MDC for area, throughput, and power.

Index Terms—FFT, DIF, DSP, R4MDC, 2-parallel

I. INTRODUCTION

The Fast Fourier Transform is a popular algorithm in the field of Digital Signal Processing. It is used extensively in digital communication specially OFDM systems, video broadcasting, speech and image processing. In particular, OFDM based WLAN (IEEE 802.11) systems use the FFT to analyze complex time domain sequences. With the advent of mobile devices like smartphones, tablets which use wireless LAN extensively and run on batteries, it is extremely important that these devices consume as little power as possible. In these types of applications, software solutions are not beneficial as the power consumption of a software solution is considerably higher than the dedicated hardwares and the latency is high for software. Thus, we need specially designed hardware to address these issues. The Radix-4 FFT is well suited for these kinds of applications as the hardware complexity of Radix-4 is lower than that of the Radix-2 architecture.

A pipelined architecture for a Radix-4 FFT is studied in this paper since it increases throughput without increasing the area significantly. There are two popular pipelined architecture for FFT: Single Delay Feedback (SDF) and Multi-path Delay Commutator (MDC). The SDF architecture was presented in [1]. In SDF, each stage has a feedback loop where some of the outputs of the butterfly are fed back to the memory of the same stage. The hardware utilization of SDF is 100%. On the other hand, in MDC, described in [2], processed samples from one stage are always passed to the next stage. The hardware utilization of MDC is 50%.

In some real-time applications such as OFDM or ultra-wide band (UWB) systems, where high throughput is a requirement, it is important to be able to process the input samples in parallel. Also, it is a challenge to process several samples

of an input sequence in parallel when they are received in parallel. Thus, parallel pipelined FFT architectures have become popular in recent times. In this paper, we propose an 8-parallel Radix-4 MDC (R4MDC) architecture, which is more power efficient than the regular Radix-4 MDC.

In [3], a novel Radix-2² SDF structure was proposed which is still very popular for FFT implementation. However, when high throughput is needed, SDF is not a suitable choice since a parallel architecture with SDF requires more area. In [4], a low power FFT architecture had been proposed for WLAN application, but it uses Radix-8 FFT structure. Thus, it is only suitable for input sequence lengths which are a power of eight. In [5], a variable length low power FFT architecture is proposed using Radix-2/4/8 algorithm for a OFDM system. It uses the SDF architecture which is less efficient for parallel architecture. In [6], a 4-parallel Radix-2⁴ FFT processor was presented for Ultra-Wide Band (UWB) applications. However, it used Multi-path Delay Feedback (MDF) which requires more hardware resources than a 4-parallel MDC architecture.

A power model has been presented for the architectures, but no power analysis was done on real hardware. In [7], a memory based FFT architecture was proposed using Radix-2³ algorithm. But, a memory based architecture is not useful for high throughput applications. A Radix-4 MDC architecture is proposed with parallel datapath in [8], but it is designed for an input of length 1024. In the paper [9], several parallel architectures were proposed for Radix-2^k. However, the designs do not have regular structure and also the paper does not present the power consumption on real hardware.

The dynamic power consumption of a CMOS circuit is given by [10],

$$P_{Dyn} = \alpha \cdot C_L \cdot V_{dd}^2 \cdot f_{clk} \quad (1)$$

where α is the switching probability, C_L is the sum of all capacitances that are being charged or discharged, V_{dd} is the supply voltage, and f_{clk} is the clock frequency of the circuit. The switching probability depends on the number of operations within a block including memory access. C_L depends on the hardware complexity of the circuit. It can be seen from Eq (1) that if the clock frequency is reduced, the dynamic power consumption will also decrease. Thus, a parallel pipeline architecture is an ideal candidate for power reduction.

In the following sections, we first present a theoretical background of the Radix-4 FFT algorithm. Then, we present

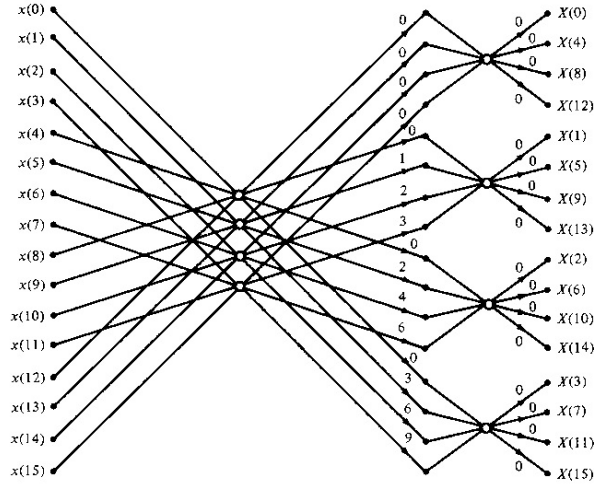


Fig. 1. Signal flow graph for 16-point Radix-4 FFT algorithm

our proposed 8-parallel Radix-4 MDC architecture and explain its operation. Then we compare our proposed design with the Radix-4 MDC architecture in terms of hardware complexity, area, throughput and power. Finally, the contribution of this paper is summarized in section VI.

II. THEORETICAL BACKGROUND

The N-point Discrete Fourier Transform (DFT) of an input sequence $x[n]$ is given below,

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk} \quad (2)$$

where $W_N^{nk} = e^{-j\frac{2\pi}{N}nk}$ is called the twiddle factor. The most common algorithm for efficient computation of the DFT when N is power of two is the Fast Fourier Transform (FFT) algorithm proposed by Cooley and Tuckey [11]. In this paper a Radix-4 Cooley-Tuckey algorithm is chosen. The signal flow graph for a 16-point Radix-4 algorithm is presented in Fig. 1. A Radix-4 FFT algorithm has $\log_4 N$ stages. Thus, the 16-point Radix-4 FFT consists of two stages. Each stage has several butterfly operations. Each butterfly consist four inputs and four outputs. Fig. 2 shows the signal flow diagram of a Radix-4 butterfly.

The Radix-4 Decimation in-Frequency (DIF) algorithm divides an N-point DFT into four $N/4$ -point DFTs. Then, each of those are divided into $N/16$ -point DFTs giving sixteen $N/16$ DFTs and so on. The final stage produces a 4-point DFT which is simply a butterfly calculation for a radix-4 FFT. The Radix-4 DIF FFT can be expressed by Eq. 3 each of which computes every fourth output sample. It can be seen that each of these consists of four summations. Each of these summations is multiplied by one of the twiddle factors ($W_N^0, W_N^N, W_N^{2N}, W_N^{3N}$).

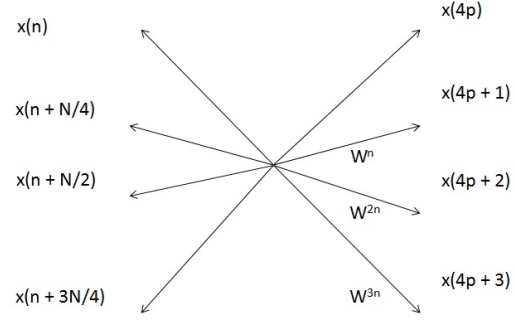


Fig. 2. Radix-4 butterfly structure

III. RADIX-4 MDC FFT ARCHITECTURE

In this section, we review the Radix-4 MDC FFT architecture described in [12]. Multipath Delay Commutator (MDC) is one of the most popular FFT architectures. Fig. 3 shows a 256-point Radix-4 MDC FFT architecture. In this architecture, input samples separated by $N/4$ (64 in this case) samples are fed to the butterfly of stage one. This can be achieved by placing three FIFO of depths 192, 128 and 64, respectively, before the first butterfly BFI.

Fig. 5 shows the internal structure of the butterflies. It consists of four adders and three multipliers. It accepts four input signals and generates four output signals, as described by Eq 3. In the complex multiplier, the samples are multiplied by the appropriate twiddle factors stored in the ROM. The lower outputs of the butterfly are fed to the delay elements of the next stage.

The switch acts as a four-port multiplexer. The internal structure of the switch consists of four multiplexers and a two-bit common control signal. Based on the control, the switch selects the appropriate input for each of its outputs. The samples are processed through the subsequent stages in the similar manner.

Overall latency for this architecture is N clock cycles, i.e. 256 cycles in our case. The hardware utilization is 50%.

IV. PROPOSED 8-PARALLEL RADIX-4 MDC FFT ARCHITECTURE

In this section, we present our proposed architecture which is capable of 8-parallel data processing. Fig. 4 show the block diagram of the proposed architecture. In this design, eight input samples are applied at the first stage in parallel. However, only two parallel datapaths are used. By efficient usage of the delay elements and data scheduling, we achieve 8-parallel processing of the samples. The hardware utilization of the design is 100% and the latency is $N/8$ clock cycles.

A. Control signals

Generating the control signals is straightforward. It can be implemented by a simple five bit counter ($b_4b_3b_2b_1b_0$). The counter is incremented by 1 in each clock cycle. The multiplexer control signals are the same across the two datapaths. They are generated as follows,

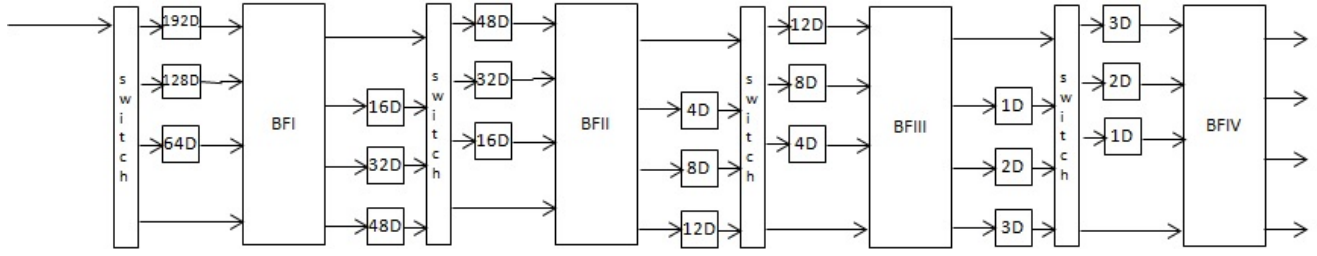


Fig. 3. 256-point R4MDC FFT architecture

$$X[4k] = \sum_{n=0}^{N/4-1} [x[n] + x[n + N/4] + x[n + n/2] + x[n + 3N/4]] W_{N/4}^{nk} \quad (3a)$$

$$X[4k + 1] = \sum_{n=0}^{N/4-1} [x[n] - jx[n + N/4] - x[n + n/2] + jx[n + 3N/4]] W_N^n W_{N/4}^{nk} \quad (3b)$$

$$X[4k + 2] = \sum_{n=0}^{N/4-1} [x[n] - x[n + N/4] - x[n + n/2] - x[n + 3N/4]] W_N^{2n} W_{N/4}^{nk} \quad (3c)$$

$$X[4k + 3] = \sum_{n=0}^{N/4-1} [x[n] + jx[n + N/4] - x[n + n/2] - jx[n + 3N/4]] W_N^{3n} W_{N/4}^{nk} \quad (3d)$$

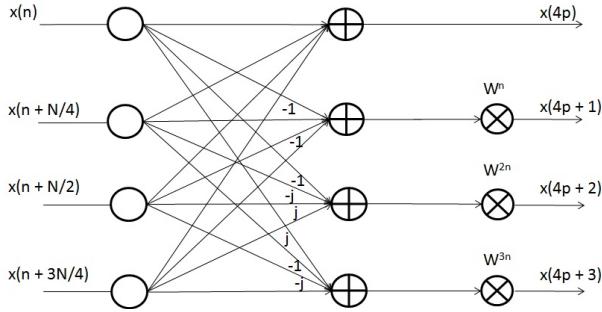


Fig. 5. Radix-4 butterfly structure

$$\begin{aligned} sel1 &= \{b_2, b_1\}; \quad sel2 = \{b_1, b_0\}; \\ sel3 &= \{b_0, b_2\}; \quad sel4 = \{b_2 b_1 + b_2 b_0 + b_2 b_1 b_0, b_0\}; \end{aligned}$$

B. ROM Address Generator

The twiddle factors are stored in the ROMs. The address generation for the ROMs is simple. We use the same 5-bit counter. However, in stage 1 different twiddle factors are selected for different datapaths. These can be easily generated too. We present below how different addresses are related. We use notation $addr_{LS}$ to denote the address for datapath L and stage S . These can be easily verified from twiddle factor sets presented in section 2.

$$addr_{21} = addr_{11} + 1;$$

V. RESULTS & DISCUSSIONS

The proposed architecture has been designed in VHDL. For comparison, we have also implemented the R4MDC as described in [12] using VHDL. The wordlength is chosen as 16. The designs are synthesized for a Xilinx FPGA using ISE 14.4 version. The target device is chosen to be Virtex-5. Although the generated outputs are in bit-reversed order, the bit reversal circuit is not considered for the discussion presented here.

In Table I, we compare the complexity and performance of the proposed 8-parallel architecture with that of R2MDC and $R2^2SDF$ described in [3]. It can be seen that the number of complex adders and complex multipliers of the proposed design is double the other two designs. However, the number of delay elements is less. The latency of the proposed architecture is one eighth of the latency of the R4MDC and $R2^2SDF$. Thus, the proposed FFT architecture greatly reduces the latency of the system. The throughput of the proposed system is increased by eight times that of the R4MDC and $R2^2SDF$.

For comparison of power consumption, we fix the throughput of all three system at the same rate and estimate the power using the Xilinx Power Analyzer tool. The synthesis results along with power estimation are presented in Table II. It can be seen that the proposed design consumes less power than the other two designs. As stated previously, the number of adders and multipliers of the proposed design are double that of the R4MDC and $R2^2SDF$. Thus, the capacitance of the circuit is roughly doubled, but since the throughput is also increased by 8, the power consumption should decrease by 75% compared

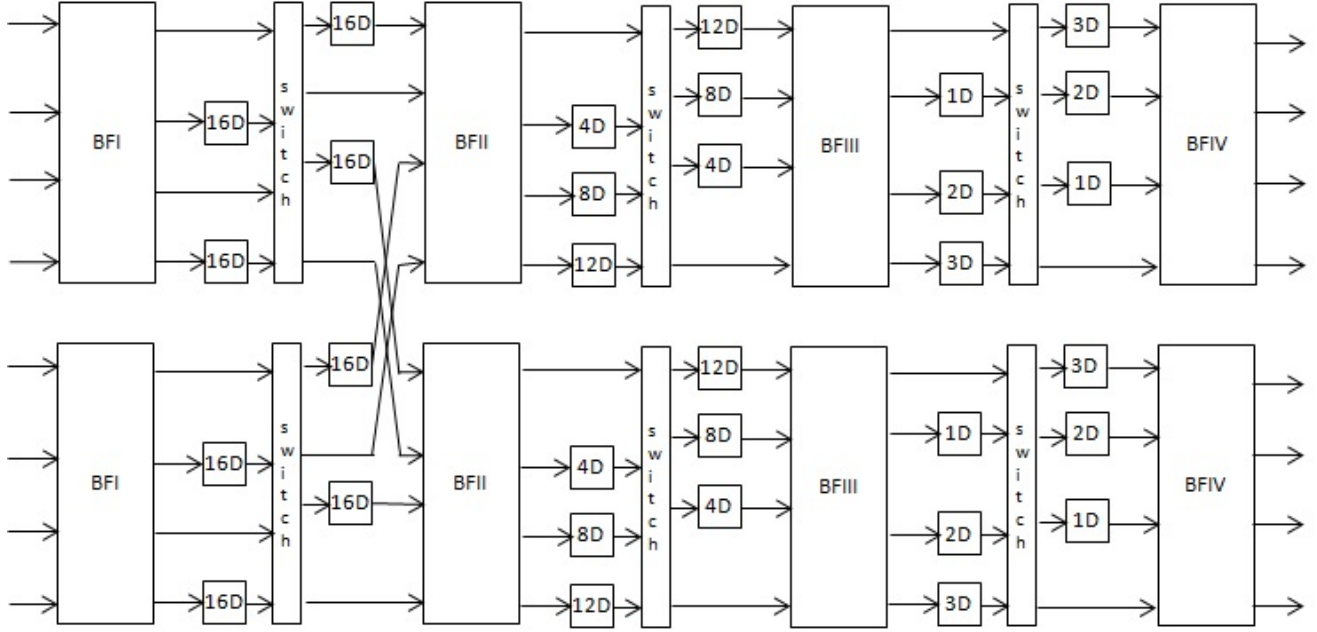


Fig. 4. Proposed 256-point 8-parallel R4MDC FFT architecture

TABLE I
COMPLEXITY AND PERFORMANCE COMPARISONS OF VARIOUS FFT ARCHITECTURES

	R4MDC	$R2^2SDF$	8-P R4MDC (Proposed)
Complex Adder	$8\log_4 N$	$4\log_4 N$	$16\log_4 N$
Complex Mult	$3(\log_4 N - 1)$	$\log_4 N - 1$	$6(\log_4 N - 1)$
Complex Memory	$5N/2 - 4$	$N - 1$	$N - 8$
Latency (cycles)	N	N	$N/8$
Throughput (samples/cycle)	1	1	8
Control	simple	simple	simple

TABLE II
AREA, LATENCY, THROUGHPUT AND POWER COMPARISON OF PROPOSED 8-PARALLEL R4MDC WITH TRADITIONAL R4MDC

	R4MDC	8-P R4MDC(Proposed)
Slice Registers	644	298
Slice LUT	1574	3582
Total Slice Used	528	1237
DSP Blocks	24	48
Frequency(Mhz)	40.87	35.76
Power(mW)	342	181

to the R4MDC according to Eq (1). However, it can be seen from the table that power reduction in FPGA is not 75%, rather it is roughly 50%. That is because the power consumption by FPGA depends on various parameters, for example, the number of slices used, number of IO banks occupied etc.

The synthesis result are consistent with our complexity analysis. The reduction in frequency is attributed to complex interconnection in FPGA and possible overhead in ROM access for four datapaths in our proposed architecture.

VI. CONCLUSION

A low power 8-parallel Radix-4 MDC FFT architecture is designed in this paper. The design was implemented in FPGA and compared with previously presented Radix-4 MDC architecture. Complexity analysis and performance analysis are performed for all three architectures. The proposed design is found to require approximately hardware resources, but at the same time it reduces the latency and increases the throughput of the system by a factor of eight.

Power analysis is performed and the proposed design is found to consume 50% less power than the traditional R4MDC. This is achieved by processing 8 samples in parallel although it uses only 2-parallel butterfly structures. Thus, this design is particularly useful for low power applications such as WLAN, etc. For future work, we plan to optimize the complex multipliers for more power reduction and optimize the ROM storage as well.

REFERENCES

- [1] E. Wold and A. Despain, "Pipeline and parallel-pipeline FFT processors for VLSI implementations," *IEEE Transactions on Computers*, vol. C-33, no. 5, pp. 414–426, 1984.
- [2] L. R. Rabinar and B. Gold, *Theory and Application of Digital Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1975.
- [3] S. He and M. Torkelson, "Design and implementation of a 1024-point pipeline FFT processor," in *Custom Integrated Circuits Conference, 1998. Proceedings of the IEEE 1998*, 1998, pp. 131–134.
- [4] C.-T. Lin, Y.-C. Yu, and L.-D. Van, "A low-power 64-point FFT/IFFT design for IEEE 802.11a WLAN application," in *2006 IEEE International Symposium on Circuits and Systems, 2006. ISCAS 2006. Proceedings*, 2006, pp. 4 pp.–4526.

- [5] Y.-T. Lin, P.-Y. Tsai, and T. D. Chiueh, "Low-power variable-length fast fourier transform processor," *Computers and Digital Techniques, IEE Proceedings* -, vol. 152, no. 4, pp. 499–506, 2005.
- [6] M. Shin and H. Lee, "A high-speed four-parallel radix-24 FFT/IFFT processor for UWB applications," in *IEEE International Symposium on Circuits and Systems, 2008. ISCAS 2008*, 2008, pp. 960–963.
- [7] S. Langemeyer, P. Pirsch, and H. Blume, "A FPGA architecture for real-time processing of variable-length FFTS," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2011, pp. 1705–1708.
- [8] D. Jeon, M. Seok, C. Chakrabarti, D. Blaauw, and D. Sylvester, "Energy-optimized high performance FFT processor," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2011, pp. 1701–1704.
- [9] M. Garrido, J. Grajal, M. Sanchez, and O. Gustafsson, "Pipelined radix-feedforward FFT architectures," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 1, pp. 23–32, 2013.
- [10] W.-C. Yeh and C.-W. Jen, "High-speed and low-power split-radix FFT," *IEEE Transactions on Signal Processing*, vol. 51, no. 3, pp. 864–874, 2003.
- [11] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [12] E. Swartzlander, W. Young, and S. Joseph, "A radix 4 delay commutator for fast fourier transform processor implementation," *IEEE Journal of Solid-State Circuits*, vol. 19, no. 5, pp. 702–709, 1984.
- [13] M. Ayinala, M. Brown, and K. Parhi, "Pipelined parallel FFT architectures via folding transformation," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 20, no. 6, pp. 1068–1081, 2012.